

YaneCode Digital

Stage d'été 2026 - YC_DS20263841

État de l'art et benchmark

Analyse des perceptions des développeurs face aux outils d'IA

Stagiaire

Charifa FAX

Domaine

Data Science, NLP et analyse de sentiment

Version

Synthèse courte orientée prototype

Date

23 juillet 2026

Cette étude ne cherche pas à couvrir toute la littérature. Elle sélectionne les travaux directement utiles aux choix du prototype : sentiment sur réseaux sociaux, textes de génie logiciel, Transformers, topic modeling, clustering et perception des assistants de code.

Table des matières

1	Contexte du sujet	2
2	Perceptions des assistants de programmation	2
2.1	Productivité	2
2.2	Expérience d'utilisation	2
2.3	Qualité et sécurité	2
3	Analyse de sentiment sur les réseaux sociaux	2
3.1	Approches lexicales et règles	2
3.2	Spécificité du génie logiciel	3
3.3	Modèles supervisés classiques	3
3.4	Transformers	3
4	Topic modeling et clustering de textes	3
4.1	LDA	3
4.2	NMF	3
4.3	K-Means	4
4.4	BERTopic	4
5	Benchmark des méthodes	4
6	Choix méthodologique retenu	5
7	Positionnement et contribution du projet	5
8	Risques méthodologiques et mesures de contrôle	5
9	Conclusion	6

1 Contexte du sujet

Les assistants basés sur les grands modèles de langage sont devenus une composante visible du travail de développement. Ils interviennent dans la recherche d'information, la génération de code, la documentation, les tests ou le débogage. Leur adoption ne signifie toutefois pas une confiance totale.

L'enquête Stack Overflow 2025 indique que 84% des répondants utilisent ou prévoient d'utiliser des outils IA dans leur processus de développement, tandis que 46% déclarent se méfier de l'exactitude des sorties contre 33% qui leur font confiance. La frustration la plus souvent citée concerne les réponses "presque correctes", qui exigent une vérification et parfois un débogage supplémentaire (STACK OVERFLOW 2025). JetBrains rapporte également une adoption large, accompagnée de préoccupations sur la qualité du code généré, la compréhension du contexte et la sécurité (JETBRAINS 2025).

Ces résultats justifient une analyse qui ne se limite pas à la polarité positive ou négative. Les thèmes de confiance, productivité, sécurité, apprentissage, emploi et contrôle humain doivent aussi être étudiés.

2 Perceptions des assistants de programmation

2.1 Productivité

Une expérience contrôlée sur GitHub Copilot a observé que le groupe disposant de l'assistant terminait une tâche JavaScript 55,8% plus rapidement que le groupe de contrôle (PENG et al. 2023). Ce résultat montre un gain potentiel dans un contexte précis ; il ne suffit pas à conclure que tous les développeurs, tâches et environnements bénéficient du même effet.

2.2 Expérience d'utilisation

Une étude avec 24 participants a montré que Copilot pouvait fournir un point de départ utile et réduire la recherche en ligne. En revanche, les participants rencontraient des difficultés pour comprendre, modifier et déboguer certaines suggestions (VAITHILINGAM et al. 2022). La perception peut donc être favorable même lorsque le gain mesuré est limité ou variable.

2.3 Qualité et sécurité

L'évaluation de scénarios de sécurité par PEARCE et al. (2022) a trouvé une proportion importante de programmes vulnérables parmi les sorties étudiées. Une autre étude utilisateur a observé que les participants assistés produisaient du code moins sûr dans plusieurs tâches et surestimaient parfois la sécurité de leur solution (PERRY et al. 2023). Ces travaux expliquent pourquoi les discussions de développeurs contiennent souvent des opinions mêlant gain de temps, méfiance et besoin de vérification.

3 Analyse de sentiment sur les réseaux sociaux

3.1 Approches lexicales et règles

VADER combine un lexique de polarité et des règles prenant en compte l'intensité, la ponctuation, les majuscules, la négation et certains contrastes. Il a été conçu pour des textes courts proches des réseaux sociaux (HUTTO et GILBERT 2014). Ses avantages sont la rapidité, l'absence d'entraînement et la possibilité d'expliquer un score.

Sa principale limite est le domaine : un mot technique peut avoir un sens différent dans un contexte logiciel. Par exemple, *kill process*, *fatal error* ou *bug* peuvent recevoir une polarité forte sans traduire directement l'état émotionnel de l'auteur.

3.2 Spécificité du génie logiciel

Les études en software engineering montrent que les outils génériques ne sont pas toujours fiables sur GitHub, Jira ou Stack Overflow. JONGELING et al. (2017) ont mis en évidence des désaccords importants entre outils et évaluations humaines. SentiStrength-SE a ensuite introduit un dictionnaire et des heuristiques propres au génie logiciel, avec de meilleurs résultats sur des commentaires Jira annotés (ISLAM et ZIBRAN 2018).

L'existence d'un corpus Stack Overflow annoté pour les émotions confirme aussi l'intérêt d'une validation humaine et de ressources adaptées au domaine (NOVIELLI et al. 2018).

3.3 Modèles supervisés classiques

Une approche classique transforme les textes en vecteurs Bag of Words ou TF-IDF, puis entraîne un classifieur comme la régression logistique ou le SVM. Elle est rapide et interprétable, mais nécessite un jeu d'entraînement dont les labels correspondent au corpus ciblé. Dans le projet actuel, les labels externes ne couvrent pas uniformément toutes les plateformes ; les utiliser comme vérité générale introduirait un biais.

3.4 Transformers

BERT préentraîne une représentation bidirectionnelle du langage, puis peut être adapté à une tâche de classification (DEVLIN et al. 2019). Pour les réseaux sociaux, TweetEval montre l'intérêt de poursuivre le préentraînement sur un corpus du même domaine et fournit un benchmark unifié pour plusieurs tâches, dont le sentiment (BARBIERI et al. 2020).

Dans le domaine logiciel, les évaluations de Transformers montrent un potentiel supérieur aux outils généralistes, mais la performance dépend fortement de la plateforme et du jeu annoté (ZHANG et al. 2020). Un Transformer reste donc une méthode à comparer, pas une vérité automatique.

4 Topic modeling et clustering de textes

4.1 LDA

LDA modélise chaque document comme un mélange de thèmes et chaque thème comme une distribution de mots (BLEI et al. 2003). La méthode est probabiliste et bien documentée, mais la qualité des thèmes dépend du prétraitement, du nombre de thèmes et de la longueur des documents.

4.2 NMF

La factorisation en matrices non négatives décompose une matrice documents-termes en représentations additives et souvent interprétables (LEE et SEUNG 1999). Associée à TF-IDF, NMF fonctionne bien pour un prototype de taille moyenne, avec un temps de calcul raisonnable et des termes dominants faciles à présenter.

4.3 K-Means

K-Means regroupe les textes représentés par TF-IDF ou embeddings. Il ne produit pas directement des “topics” ; il produit des groupes qu’il faut interpréter à partir des termes et documents les plus proches du centre. Sa simplicité en fait une bonne méthode de comparaison avec NMF.

4.4 BERTopic

BERTopic combine embeddings de Transformers, clustering et une variante de TF-IDF par classe afin de générer des représentations thématiques (GROOTENDORST 2022). Il peut produire des thèmes sémantiques de qualité, mais il ajoute des dépendances, du temps de calcul et plusieurs paramètres. Il est pertinent comme extension, mais non indispensable pour respecter le délai du prototype.

5 Benchmark des méthodes

Méthode	Points forts	Limites	Décision pour le projet
VADER	Très rapide ; aucun entraînement ; score explicable ; adapté aux textes sociaux.	Lexique générique ; erreurs possibles sur jargon technique, sarcasme et textes longs.	Retenu comme base-line.
TextBlob	Facile à utiliser ; bonne valeur pédagogique.	Modèle générique, moins adapté au domaine logiciel ; faible valeur ajoutée par rapport à VADER.	Présenté dans le benchmark, non prioritaire.
TF-IDF + Logistic Regression/SVM	Rapide ; interprétable ; bonne baseline supervisée.	Nécessite des labels homogènes et fiables.	Option après constitution d’un échantillon annoté plus large.
SentiStrength-SE	Adapté au vocabulaire du génie logiciel.	Outil plus ancien ; intégration et maintenance moins simples qu’un pipeline Python moderne.	Référence méthodologique et point de comparaison.
Transformer social-media	Contexte mieux capturé ; bon point de comparaison moderne.	Coût supérieur ; limite de longueur ; biais du domaine d’entraînement.	Retenu comme deuxième méthode.
LDA	Modèle probabiliste classique ; littérature abondante.	Sensible aux paramètres ; thèmes parfois moins nets sur textes courts.	Option de comparaison si le temps le permet.
NMF + TF-IDF	Rapide ; reproductible ; thèmes faciles à interpréter.	Basé sur le vocabulaire ; compréhension sémantique limitée.	Retenu pour les thèmes.
K-Means + TF-IDF	Simple ; répond à l’existence de clustering.	Nombre de groupes imposé ; groupes à interpréter manuellement.	Retenu pour comparer au topic modeling.
BERTopic	Embeddings sémantiques ; gestion moderne des thèmes.	Installation, mémoire et réglages plus coûteux.	Travaux futurs, sauf temps disponible.

6 Choix méthodologique retenu

Compte tenu du cahier des charges, de la taille du corpus et du délai, le pipeline suivant offre un équilibre entre qualité, démonstration et faisabilité :

1. nettoyage différencié selon les méthodes, sans écraser le texte brut ;
2. EDA globale et par plateforme ;
3. VADER comme baseline sentiment ;
4. Transformer préentraîné comme deuxième méthode ;
5. échantillon humain stratifié pour évaluer les deux méthodes ;
6. TF-IDF + NMF pour les thèmes ;
7. TF-IDF + K-Means pour le clustering ;
8. dashboard présentant les distributions, les tendances et des exemples représentatifs.

7 Positionnement et contribution du projet

Les travaux consultés étudient soit l'efficacité d'un assistant particulier, soit la sécurité du code généré, soit la détection de sentiment sur une plateforme de génie logiciel. Le présent prototype se positionne comme une étude exploratoire multi-plateforme. Sa contribution attendue n'est pas un nouveau modèle de pointe, mais un pipeline reproductible permettant de :

- comparer plusieurs espaces de discussion de développeurs ;
- relier polarité, thèmes et outils IA mentionnés ;
- rendre visibles les écarts entre adoption, productivité perçue, confiance et inquiétudes ;
- documenter clairement les biais de collecte et les limites des modèles.

Cette contribution doit rester formulée avec prudence : le corpus est observationnel, non aléatoire et déséquilibré entre plateformes.

8 Risques méthodologiques et mesures de contrôle

Risque	Mesure retenue
Sentiment mal interprété	Validation humaine et analyse des erreurs.
Jargon technique	Comparaison baseline/Transformer et lecture d'exemples.
Une source domine les résultats	Résultats globaux accompagnés d'analyses par plateforme.
Dates manquantes	Exclusion des documents non datés des conclusions temporelles.
Labels externes incompatibles	Conservation comme métadonnées séparées, sans fusion automatique.
Surinterprétation des topics	Lecture des documents représentatifs et nommage manuel.
Données publiques mais personnelles	Analyse agrégée ; pas de profilage des auteurs.

9 Conclusion

La littérature justifie une approche comparative et prudente. VADER fournit une baseline simple ; les Transformers apportent une meilleure prise en compte du contexte ; les outils spécifiques au génie logiciel rappellent l'importance du domaine. Pour les thèmes, NMF et K-Means répondent au besoin de faisabilité et d'interprétation, tandis que BERTopic reste une amélioration possible.

Le choix final n'est donc pas la méthode la plus complexe, mais le plus petit ensemble de méthodes permettant une comparaison crédible, une validation humaine et une démonstration claire.

Références

- BARBIERI, Francesco et al. (2020). "TweetEval : Unified Benchmark and Comparative Evaluation for Tweet Classification". In : *Findings of the Association for Computational Linguistics : EMNLP 2020*, p. 1644-1650. DOI : [10.18653/v1/2020.findings-emnlp.148](https://doi.org/10.18653/v1/2020.findings-emnlp.148).
- BLEI, David M., Andrew Y. NG et Michael I. JORDAN (2003). "Latent Dirichlet Allocation". In : *Journal of Machine Learning Research* 3, p. 993-1022.
- DEVLIN, Jacob et al. (2019). "BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding". In : *Proceedings of NAACL-HLT*, p. 4171-4186. DOI : [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423).
- GROOTENDORST, Maarten (2022). "BERTopic : Neural Topic Modeling with a Class-based TF-IDF Procedure". In : *arXiv preprint arXiv :2203.05794*. DOI : [10.48550/arXiv.2203.05794](https://doi.org/10.48550/arXiv.2203.05794).
- HUTTO, C. J. et Eric GILBERT (2014). "VADER : A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text". In : *Proceedings of the International AAAI Conference on Web and Social Media*. T. 8. 1, p. 216-225. DOI : [10.1609/icwsm.v8i1.14550](https://doi.org/10.1609/icwsm.v8i1.14550).
- ISLAM, Md Rakibul et Minhaz F. ZIBRAN (2018). "SentiStrength-SE : Exploiting Domain Specificity for Improved Sentiment Analysis in Software Engineering Text". In : *Journal of Systems and Software* 145, p. 125-146. DOI : [10.1016/j.jss.2018.08.030](https://doi.org/10.1016/j.jss.2018.08.030).
- JETBRAINS (2025). *The State of Developer Ecosystem in 2025*. URL : <https://devecosystem-2025.jetbrains.com/> (visité le 23/07/2026).
- JONGELING, Robbert et al. (2017). "On Negative Results When Using Sentiment Analysis Tools for Software Engineering Research". In : *Empirical Software Engineering* 22, p. 2543-2584. DOI : [10.1007/s10664-016-9493-x](https://doi.org/10.1007/s10664-016-9493-x).
- LEE, Daniel D. et H. Sebastian SEUNG (1999). "Learning the Parts of Objects by Non-negative Matrix Factorization". In : *Nature* 401, p. 788-791. DOI : [10.1038/44565](https://doi.org/10.1038/44565).
- NOVIELLI, Nicole, Fabio CALEFATO et Filippo LANUBILE (2018). "A Gold Standard for Emotion Annotation in Stack Overflow". In : *arXiv preprint arXiv :1803.02300*. DOI : [10.48550/arXiv.1803.02300](https://doi.org/10.48550/arXiv.1803.02300).
- PEARCE, Hammond et al. (2022). "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions". In : *2022 IEEE Symposium on Security and Privacy*, p. 754-768. DOI : [10.1109/SP46214.2022.9833571](https://doi.org/10.1109/SP46214.2022.9833571).
- PENG, Sida et al. (2023). "The Impact of AI on Developer Productivity : Evidence from GitHub Copilot". In : *arXiv preprint arXiv :2302.06590*. DOI : [10.48550/arXiv.2302.06590](https://doi.org/10.48550/arXiv.2302.06590).
- PERRY, Neil et al. (2023). "Do Users Write More Insecure Code with AI Assistants?" In : *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. DOI : [10.1145/3576915.3623157](https://doi.org/10.1145/3576915.3623157).
- STACK OVERFLOW (2025). *2025 Developer Survey : Artificial Intelligence*. URL : <https://survey.stackoverflow.co/2025/ai> (visité le 23/07/2026).

- VAITHILINGAM, Priyan, Tianyi ZHANG et Elena L. GLASSMAN (2022). "Expectation vs. Experience : Evaluating the Usability of Code Generation Tools Powered by Large Language Models". In : *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, p. 1-7. DOI : [10.1145/3491101.3519665](https://doi.org/10.1145/3491101.3519665).
- ZHANG, Ting et al. (2020). "Sentiment Analysis for Software Engineering : How Far Can Pre-trained Transformer Models Go ?" In : *2020 IEEE International Conference on Software Maintenance and Evolution*.